

Michael — how to run 0195, and what was actually wrong this time

Date: 22 August 2026 **Short answer:** you were not doing anything wrong. The file still had a hazard in it. It is gone now.

1. Answering your questions directly

"Should I try running it without RLS enabled?"

No — please don't. It would not help, and it would cost you something real.

It would not help because the error you are getting, `42P01: relation "a" does not exist`, is a **parser** error. Postgres is complaining that it cannot find a table called `a` while it is still reading your text. That happens *before* any security rule is consulted. RLS is not involved in the failure.

It would cost you something because RLS is exactly what keeps employee Social Security numbers from being readable. Turning it off to get a migration to run would trade a real protection for an imaginary fix.

"Is this a Supabase issue?"

Partly yes — and you are not the only one. I researched this the way you asked, and here is what I found.

Someone hit **your exact error** in the Supabase SQL editor on 12 August 2026 — ten days ago — with completely unrelated SQL. Their words could be yours:

*"The weird part is that the SQL file **doesn't contain any table/relation named a** ... What confuses me most is that **Supabase doesn't give me a line number or point to the statement causing the error.**"*

(Stack Overflow question 79993483)

There is also a **confirmed bug in Supabase's own statement splitter**. Supabase does not simply hand your whole file to Postgres; it chops it into individual statements first, and that chopping code has a documented history of getting it wrong. From the Supabase CLI issue tracker:

"The CLI's SQL statement splitter mis-parses migrations... The splitter loses track of dollar-quote nesting depth, treats statement-terminating semicolons inside the outer block as top-level boundaries, and bundles multiple separate statements into a single string."

(supabase/cli issue #5146; related bugs #4746, #5020, #5062)

So the pattern is real and documented: **the splitter can lose its place, and when it does, the error it reports points nowhere useful**. That is exactly what you experienced — the same error on two machines, with no line number.

But I am not going to blame Supabase and stop there, because that would leave you stuck. The honest split of responsibility is this: their splitter is fragile, *and my file gave it something to trip on*. I can only fix my half, so I did.

2. What was actually still wrong in the file

I proved the SQL itself is fine. Applied to a real PostgreSQL 15 server, behind a real run of all 194 earlier migrations, the editor-safe 0195 applies with a clean exit on a **first** application. So the file is valid SQL. Something was damaging it **in transit**, between your clipboard and the database.

Then I found the specific thing, and there was **exactly one** of it in the whole file. Buried in a descriptive note attached to the SSN column was this sentence:

*"Never select this **into a** list view."*

Read that as a computer reading SQL. `into a` — the word `into` is a SQL keyword, and `a` is the name that follows it. If the splitter loses track of the fact that this sentence is quoted text and not code, it sees an instruction referring to **a table named a**. And the error it produces is, character for character, **your error**.

I searched the entire file for anything that could produce that error. There was one site. That sentence.

I also removed two other things that make a splitter lose its place to begin with:

- **An em dash (—).** One non-standard character in the file, sitting in that same SSN note. Special characters like this are the cheapest way for text to get corrupted as it moves between programs.
- **Semicolons inside quoted sentences.** A semicolon is how SQL says "statement over". Two of my descriptive notes contained one mid-sentence, e.g. "Revoke it; ssn_last_four is what screens should read." A splitter that ignores quotes cuts the file in half right there.

The wording was changed — same meaning, plain full stops, no special characters. **All three hazard counts are now zero, and a test enforces that they stay zero.**

To be sure the test is not decoration, I put the bad sentence **back** and re-ran: three tests failed immediately. Then I removed it again and all fourteen passed. The guard works.

3. Please run this file

```
supabase/migrations/editor-safe/0195_employee_payroll_setup.EDITOR_SAFE.sql
```

Pull the latest from the repo first, because this file changed today.

How to paste it:

1. Open the file and **select all, copy** — do not retype any part of it.
2. In the Supabase SQL editor, **clear the box completely** before pasting. A leftover fragment from a previous attempt is its own source of confusion.
3. Paste, and run **once**.
4. Expect a short list of green `NOTICE` lines mentioning policies that "does not exist, skipping". **Those are normal and mean it worked** — the file tidies up old rules before creating new ones, and on a first run there is nothing to tidy.

If it still fails, do this and it will take two minutes to pinpoint:

- Run it in **two halves**. Paste lines 1-210 and run. Then paste the rest and run. The half that fails tells us instantly where the splitter is losing its place, which is the one thing I have not been able to observe from here.
- Send me the error **and** which half produced it.

4. What I verified before asking you to run it again

I am telling you this because last time I said "proved sound" and you were still blocked, and I do not want to repeat that.

- **It applies on a FIRST run.** All 194 earlier migrations were applied to a fresh database, then 0195 — clean exit. The first run is the only kind you ever perform, so that is the only test that counts.
- **The SSN protection actually works this time.** After a single run, `ssn_last_four` is readable and the full SSN is **not**. That is the ordering bug from the last round, now confirmed fixed on a first application.
- **The comment-free copy and the full original build an identical database.** Two separate databases compared object by object: **491 items against 491, no differences**. The audit function's logic is identical in both once comments are set aside (2,344 characters each way).
- **The audit function reports zero problems** after the run.
- **Full test suite: 7,879 tests passing.** Type check clean. No new lint errors.

5. If Supabase fights you again

You have a second route that avoids the SQL editor's splitter completely, and therefore avoids this entire class of problem. From your Mac, in the project folder:

```
supabase db push
```

That sends migrations over a database connection rather than through the web editor. It is also how the file was tested here. If

the editor ever gives you an error with no line number again, this is the way around it — and if you would like, I will write you a short step-by-step for setting it up.

In one paragraph

You were not doing anything wrong, and the answer to "should I turn off RLS" is no — the error happens while Postgres is still reading the text, long before security rules matter, so turning them off would give up real protection for nothing. Supabase's statement splitter is genuinely fragile and there is a documented bug plus another user hitting your exact error ten days ago, but my file also handed it something to trip on: one sentence reading "select this **into a** list view", which a confused parser reads as a table named `a`. That sentence, one em dash, and two mid-sentence semicolons are gone, every hazard count is zero, a test keeps them at zero, and the file is proven to apply cleanly on a first run and to build a database identical to the original.